Code

Ben Wollack

2026-05-28

Table of contents


```
# read in packages
library(tidyverse)
library(here)
library(janitor)
library(lubridate)
library(showtext)
library(treemap)
library(viridis)
library(ggridges)

# read in my data and store it as an object called ben-data
ben_data <- read.csv(here("data", "ben-data-tracker - Sheet1.csv"))
```

Data Cleaning

```
# create new object called ben_clean from ben_data
ben_clean <- ben_data |>
  # remove capital letters, add underscores
  clean_names() |>
  # convert dates into an ordered variable
  mutate(date_clean = ymd(date)) |>
  # set day of the week to have levels by chronological order
  mutate(day_of_week = as_factor(day_of_week),
         day_of_week = fct_relevel(day_of_week,
                                   "Monday",
                                   "Tuesday",
                                   "Wednesday",
                                   "Thursday",
                                   "Friday",
                                   "Saturday",
                                   "Sunday")) |>
  # remove unneeded columns
  subset(select = -date)

# check data frame's structure to ensure columns are read correctly
str(ben_clean)
```

```
'data.frame':   32 obs. of  7 variables:
 $ distance_walked_km: num  8.03 6.23 4.49 4.29 7.04 ...
 $ day_of_week       : Factor w/ 7 levels "Monday","Tuesday",..: 4 5 6 7 1 2 3 4 5 6 ...
 $ bus_trips         : int  3 0 0 0 0 2 2 3 2 0 ...
 $ calendar_events   : int  5 4 1 0 3 6 4 4 4 1 ...
 $ high_temperature_c: int  22 21 16 18 19 21 21 22 19 18 ...
 $ lunch_location    : chr  "IV" "Campus" "IV" "Campus" ...
 $ date_clean        : Date, format: "2026-04-23" "2026-04-24" ...
```

```
# add custom fonts
font_add_google("Darker Grotesque", "grot")
font_add_google("Lexend", "lex")
# use showtext to render fonts
showtext_auto()
```

Day of week and date now have associated number values, so the data can be ordered.

Each figure will use different columns, so I will begin each plot with a select function.

Figure 1 - Timeseries

Variables being used: distance walked, date, day of week

The aim of this time series is to provide a general summary of how much I walked each day over the course of data collection. Furthermore, I highlighted each day of the week to see if any patterns emerged (for example, if there was difference between on walking on weekdays vs weekends).

```
# create new object to only include needed columns
ben_time <- ben_clean |>
  # select needed columns
  select(date_clean, distance_walked_km, day_of_week)
```

```
# add base layer using cleaned data for this chart
# x-axis: date
# y-axis: distance walked
# color: by day of week (for points)
ggplot(data = ben_time,
       mapping = aes(
         x = date_clean,
         y = distance_walked_km,
         color = day_of_week
       )) +
  # add the distance walked information
  geom_line(
    # use the distance walked column
    mapping = aes(y = distance_walked_km),
    # grey colored line
    color = "grey10",
    # adjust the line's width
    linewidth = 0.3
      ) +
  # add points
  geom_point() +
  # color points in rainbow order by day of week
  scale_color_manual(values = c(
    "Monday" = "red",
    "Tuesday" = "orange2",
    "Wednesday" = "yellow",
    "Thursday" = "green4",
    "Friday" = "cyan3",
```

```
  "Saturday" = "blue1",
  "Sunday" = "purple3"
)) +
# adjust y-axis to start at 0
ylim(0, 14) +
# adjust color of the background and panel grid to be the same
# hides grid
theme(
  panel.background = element_rect(fill = "#D1BCA1"),
  panel.grid = element_line(color = "#D1BCA1")) +
# creating title
# renaming axes
# renaming legend (mapped to color)
labs(
  x = "Date",
  y = "Distance Walked (km)",
  color = "Day Of Week",
  title = "Daily Distance Walked"
) +
# add an annotation
annotate(
  # annotation uses curve geometry
  geom = "curve",
  # sets the beginning x location of the curve (must use date format to
  # align with chart)
  x = as.Date("2026-05-08"),
  # sets the beginning y location of the curve
  y = 11.25,
  # sets the end x location of the curve
  xend = as.Date("2026-05-15"),
  # sets the end y location of the curve
  yend = 13.13,
  # sets direction of curve
  curvature = -0.5,
  # adjusts arrow length
  arrow = arrow(length = unit(4, "mm"))) +
# add text annotation
annotate(
  # use text geometry
  geom = "text",
  # x location of text
  x = as.Date("2026-05-08"),
```

```
  # y location of text
  y = 11,
  # actual text
  label = "Hiking Day",
  # center text around set x location
  hjust = "center",
  # use larger size
  size = 8,
  # use custom font
  family = "grot") +
# annotation 2, same format as before, so fewer annotations
# curve and arrow geometry
annotate(
  geom = "curve",
  x = as.Date("2026-05-17"),
  y = 2.5,
  xend = as.Date("2026-05-11"),
  yend = 3.33,
  curvature = 0.5,
  arrow = arrow(length = unit(4, "mm"))) +
annotate(
  geom = "text",
  x = as.Date("2026-05-17"),
  y = 1,
  # actual text with line break
  label = "Less walking \non Sundays (usually)",
  hjust = "center",
  size = 5,
  family = "grot")
```

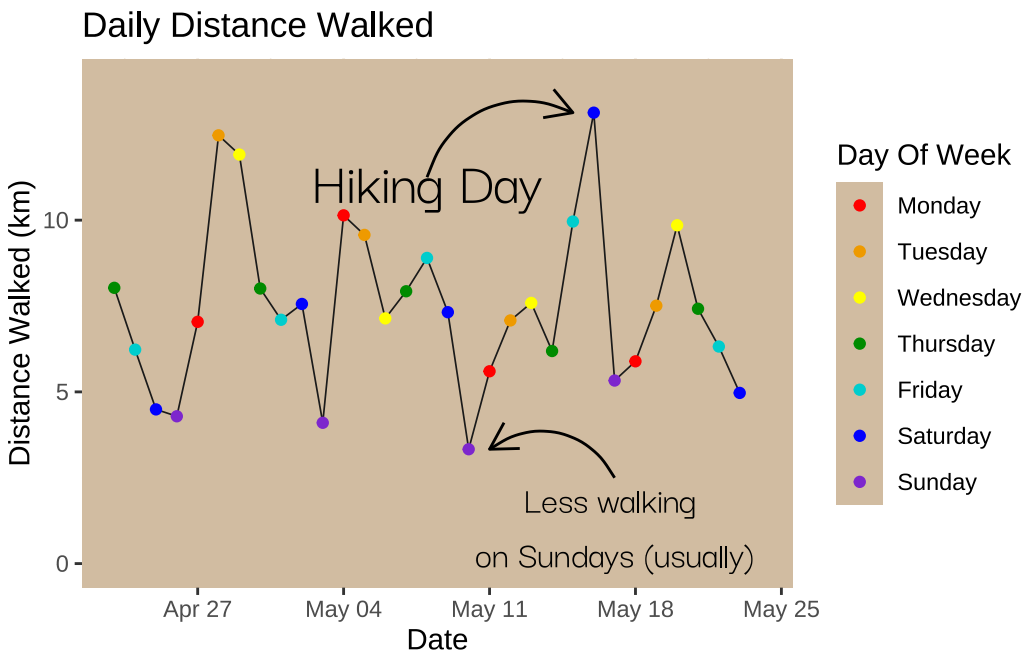


Figure 2 - Treemap

Variables being used: distance walked, bus trips, lunch location

Although distance walked dictates the size of each box, the time series shows that each day does not differ too much in terms of distance walked, and this chart also shows that the sizes are more or less homogeneous when averaged out, visually showing that lunch location and bus trips did not alter distance walked that much. What is interesting is how often I ate lunch on campus. I am busy after all, and that meal plan sure comes in handy.

```
# create new object to only include needed columns
ben_tree <- ben_clean |>
  # select needed columns
  select(distance_walked_km, bus_trips, lunch_location)
```

```
# create a treemap diagram
# use ben_tree for the data
treemap(ben_tree,
          # hierarchy of variables to display
          index = c("lunch_location", "bus_trips", "distance_walked_km"),
          # box size controlled by distance walked
          vSize = "distance_walked_km",
```

```
        # creates plot title
        title = "Distance Walked (km) by Lunch Location and Bus Trips",
        fontsize.title = 12,
        # sets the color palette based on layer 1 (lunch location)
        palette = c("#2F2ABF", "#2C6B36", "#C21D1D", "#BFAB1B"),
        # sets border colors for the 3 layers
        # black for first 2, grey for 3rd
        border.col=c("black", "black", "grey50"),
        # adjust border widths, higher layers are thicker
        border.lwds=c(4,2.5,1),
        # adjusts font size on labels, 3rd layer has no labels since size already
        # accounts for the value
        fontsize.labels=c(15, 12, 0),
        # sets layer 1 text to black and layer 2 to white
        fontcolor.labels=c("black", "white"),
        # removes highlighting on labels
        bg.labels=c("transparent"),
        # layer 1's labels are centered, layer 2 in top left of box
        align.labels=list( c("center", "center"), c("left", "top")),
        fontfamily.labels = "lex",
)
```

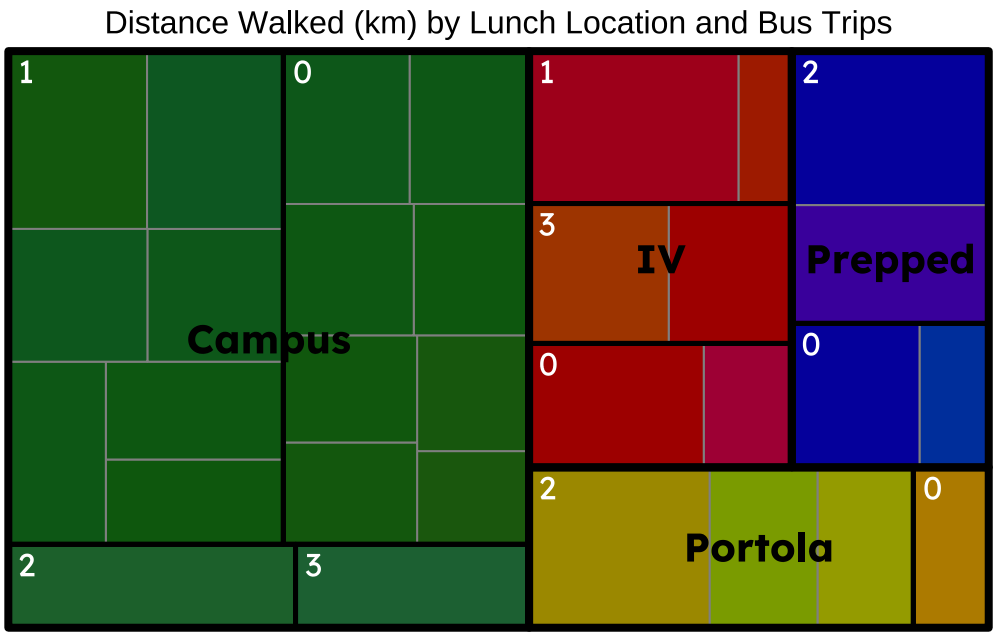


Figure 3 - Scatterplot

Variables used: distance_walked, day_of_week, calendar_events

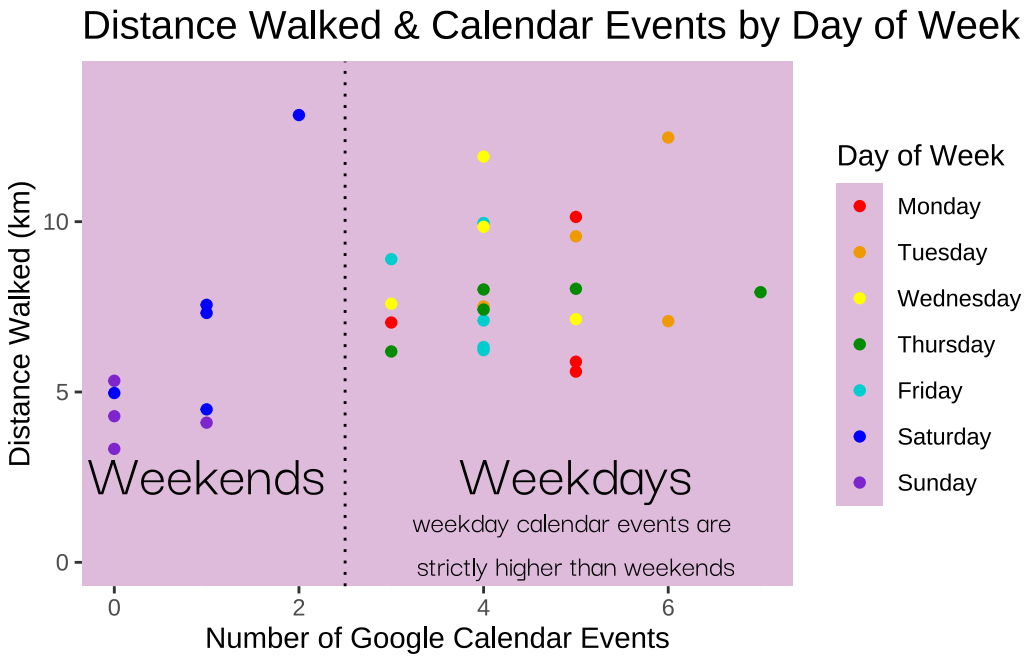
The scatterplot finally has a clear pattern emerging. There is a small pattern of me walking less on Sundays, which the time series already illustrated, but even more striking is how many fewer calendar events I had on weekends. This makes sense in context, but I am surprised by how clear it is on the chart. Aside from Memorial Day (which is essentially a weekend), weekends have strictly fewer calendar events than weekdays (absolutely no exceptions here). This pattern is notable enough that I drew attention to it.

```
# create new object to only include needed columns
ben_scatter <- ben_clean |>
  # select needed columns
  select(distance_walked_km, day_of_week, calendar_events)
```

```
# create a base layer for a scatterplot
# data: use ben_scatter
# x-axis: calendar events
# y-axis: distance walked
# color: group by day of week
ggplot(
  data = ben_scatter,
  mapping = aes(
    x = calendar_events,
    y = distance_walked_km,
    color = day_of_week
  )) +
  # create scatterplot
  geom_point() +
  # adjust colors, same as figure 1
  scale_color_manual(values = c(
    "Monday" = "red",
    "Tuesday" = "orange2",
    "Wednesday" = "yellow",
    "Thursday" = "green4",
    "Friday" = "cyan3",
    "Saturday" = "blue1",
    "Sunday" = "purple3"
  )) +
  # adjust background color and remove panel gridlines
  theme(
```

```
  panel.background = element_rect(fill = "#DEBADB"),
  panel.grid = element_line(color = "#DEBADB"),
  plot.title = element_text(size = 15)) +
# rename axes and legend, create title
labs(
  x = "Number of Google Calendar Events",
  y = "Distance Walked (km)",
  title = "Distance Walked & Calendar Events by Day of Week",
  color = "Day of Week"
) +
# adjust y-axis limits so it starts at 0
ylim(0, 14) +
# annotate with a vertical line to highlight weekday vs weekend pattern
# xintercept sets the location
# color sets the color to black
# linetype makes it dotted
geom_vline(
  xintercept = 2.5,
  color = "black",
  linetype = "dotted"
) +
# since I annotated these in detail in figure 1, I'll make them sparse here
# creating the weekend annotation
annotate(
  geom = "text",
  x = 1,
  y = 2.5,
  label = "Weekends",
  hjust = "center",
  size = 8,
  family = "grot") +
# create annotation for weekday label
annotate(
  geom = "text",
  x = 5,
  y = 2.5,
  label = "Weekdays",
  hjust = "center",
  size = 8,
  family = "grot") +
# create annotation describing the pattern
annotate(
```

```
    geom = "text",
    x = 5,
    y = 0.5,
    # line break needed
    label = "weekday calendar events are \nstrictly higher than weekends",
    hjust = "center",
    size = 4,
    family = "grot")
```



Final Write Up

Visualization

For visualization 1, I tried to parse out patterns, and I couldn't really find any aside from a few points of interest (such as the day I went on a long hike, which drove up the walked distance for that day). Highlighting these points required using annotations, so I added that to the plot.

For visualization 2, there also aren't that many patterns. The main pattern is how often I ate lunch at each location rather than distance walked, and bus trips didn't have any noteworthy patterns either. To highlight lunch location, I attached color to this variable to make it stand

out. I also made the borders thick and used a different font that naturally looks bold while pushing other information off to the side by using less striking fonts and borders.

For visualization 3, there was a clear pattern between calendar events for weekdays and weekends (not so much for distance walked though), so I highlighted this using a vertical line annotation and accompanying text.

For more general visualization aspects, I stuck with bold colors for the data to make it easy to read while using pale colors for the backgrounds. I also chose fonts that were easy to read but not too overwhelming (with the exception of visualization 2, which needed a bolder font to match with a dark background and a noisy dataset.

Sources

Cleaning the data involved making the categorical variables `date` and `day of week` ordered variables. For days of the week, this was simple, and I used as_factor() as done in workshop. For dates, it was a bit trickier, and I had to bring in a new, similar function that could read a date as year-month-day data. During cleaning, I also had to load in external fonts.

For visualization 1, inspiration mostly came from class. Initial inspiration came from the Himalayan Expeditions dataset (Gregorios Karamani) since I wanted to also plot temperature. I ended up forgoing this as it required a secondary axis (different variable with different units) that would've made the graph trickier to read. The hardest part for visualization 1 was working with an axis in date format. This affected annotation placement (since I had to specify x-axis location in terms of the date variable).

For visualization 2, inspiration came from Data to Viz. I initially wanted Circular Packing (Yan Holtz), but I was having trouble getting the packages to run, and I ended up deciding the Treemap (also Yan Holtz) would've worked better given the more efficient use of space. I ended up using treemap() rather than treemapify() even though it didn't use ggplot since I liked the aesthetics on the treemap() chart more. Surprisingly, it also required less wrangling.

For visualization 3, inspiration also came from class and Data to Viz (Yan Holtz). Specifically, I wanted the group scatterplot, where points were labeled with different colors to represent a third variable. This visualization was definitely the most straightforward, although I had to find a new annotation that would as a line rather than a curve to highlight the weekend vs weekday pattern.

I used template code from Data to Viz. Stack Overflow and Google AI helped a lot with debugging. Prompts were mostly asking Google how to do a certain action (for example, how to get RStudio to read dates as a date variable rather than a categorical variable).